南京郵電大學
Nanjing University of Posts and Telecommunications

Python编程与人工智能应用

py for AI

张伯雷
南京邮电大学，计算机学院

https://bolei-zhang.github.io/course/python-ai.html

2026/4/20

目录

- 线性回归问题简介
- 单变量线性回归问题
- 基于Scikit-learn库求解单变量线性回归
- 自定义求解单变量线性回归
  - 基于最小二乘法
  - 基于梯度下降法
- 多变量线性回归问题

人工智能 vs 机器学习

- 符号主义（Symbolism）：
  - ✓ 一种基于逻辑推理的智能模拟方法
  - ✓ 又称逻辑主义、心理学派或计算机学派。
  - ✓ 启发式算法→专家系统→知识工程，知识图谱

- 连接主义（Connectionism）：
  - ✓ 又称仿生学派或生理学派
  - ✓ 是一种基于神经网络及网络间的连接机制与学习算法的智能模拟方法，比如深度学习
  - ✓ 从历史经验中去学习

- 行为主义（Actionism）：
  - ✓ 强化学习：“感知——行动”的行为智能模拟方法

机器学习

- 机器学习“预测”场景
  - ✓ 根据细风和晚霞预测明天的气温和天气
  - ✓ 根据餐馆的座位情况和香味预测这家店的菜品
  - ✓ 根据地铁、学区、居室情况估算房价

$$\boxed{y} = \boxed{f}(\boxed{X})$$

模型

预测值

特征数据

- 机器学习
  - ✓ 让计算机模仿人类，从过去经验中学习一个“模型”，通过学到的模型再对新情况给出一个预测
  - ✓ “经验”通常是以“数据”的形式存在

- 监督学习
  - ✓ 分类（classification）：预测的值是“离散”的
  - ✓ 回归（regression）：预测的值是“连续”的

问题定义

- 数据集
- 训练数据集(X_train, y_train)

特征 feature (d列)

标签 label

样本 instance (m行)

x_1	x_2	x_3	x_4	x_5	…	y
1.2	100	200	3	12		5.2
3.5	200	35	12	4.6		7.8
4.5	7.2	100	5	13.5		9.2
…	…	…	…	…		…

- 测试数据集X_test

x_1	x_2	x_3	x_4	x_5	…	y
1.8	200	20	4	13		?
13.5	300	15	11	4.8		?
14.5	52	2	3	12.5		?

- 目标
  - ✓ 从训练数据集中学习模型f（如何形式化f）
  - ✓ 使得在测试数据集中$f(X_test)$和真实的y_test尽量接近（如何定义接近）

线性回归问题与模型

- 基本形式
  - ✓ 给定有d个属性的样例 $\boldsymbol{x}=(x_1;x_2;x_3;\ldots;x_d)$，其中 x_i 是 $\boldsymbol{x}$ 在第 i 个属性上的取值，线性回归试图学习得到一个预测函数 $f(\boldsymbol{x})$，该函数的值是各属性值的线性加权和，公式如下

$$f(\boldsymbol{x})=w_1x_1+w_2x_2+\cdots+w_dx_d+b$$

- 向量形式为：$f(\boldsymbol{x})=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b$
  - ✓ $\boldsymbol{w}$ 和 b 是**可学习和调整的参数**，**可根据经验手动设定，或自动从数据中学习获得**
  - ✓ 预测某套商品房的总价

$$f(\boldsymbol{x})=3\times\text{面积大小}+0.5\times\text{楼层指数}+0.2\times\text{卧室数量指数}$$

- 单变量线性回归与多变量线性回归

单变量线性回归问题

- 当样本仅1个属性时（即只有 x_1），只要求解两个参数（w_1和 b），是单变量线性回归模型

$$f(\boldsymbol{x}) = w_1 x_1 + b$$

- 案例描述：设某小区通过某房产中介处已售出5套房，房屋总价与房屋面积之间有如下的数据关系。现有该小区的一位业主想要通过该房产中介出售房屋，在业主报出房屋面积后，根据训练数据，中介能否能估算出该房屋的合适挂售价格?

训练样本	房屋面积（平方米）	房屋总价（万元）
1	75	270
2	87	280
3	105	295
4	110	310
5	120	335

案例分析

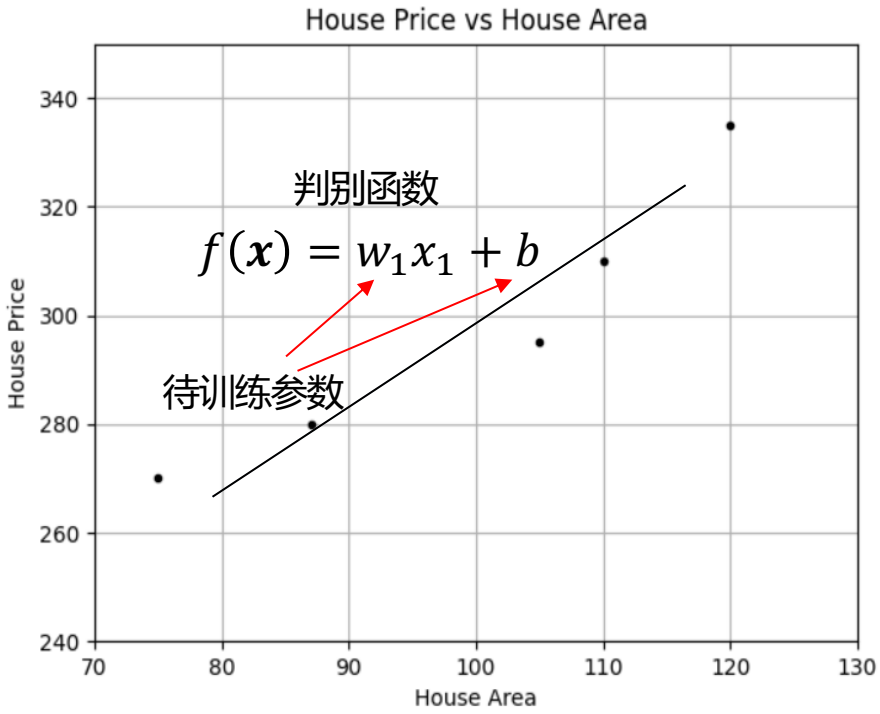
- 把房屋面积看成自变量 x，房屋总价看成因变量 y，先通过绘图看出二者之间的关系

```
#代码3.1 查看小区已售房屋房价样本数据
import numpy as np
import matplotlib.pyplot as plt
def initPlot():
    plt.figure()
    plt.title('House Price vs House Area')
    plt.xlabel('House Area') #设置x轴标题
    plt.ylabel('House Price') #设置y轴标题
    # 设置x轴和y轴的值域分别为70~130和240~350
    plt.axis([70, 130, 240, 350])
    plt.grid(True) #显示网格
    return plt
plt = initPlot()
xTrain = np.array([75, 87, 105,110,120])
yTrain = np.array([270,280,295,310,335])
plt.plot(xTrain, yTrain, 'k.') #k表示绘制颜色为黑色，点表示绘制散点图
plt.show()
```

案例分析

- 把房屋面积看成自变量 x，房屋总价看成因变量 y，先通过绘图看出二者之间的关系

1. 房屋总价随着房屋面积的变化，大致呈现线性变化趋势；
2. 如果根据现有的训练样本数据能够拟合出一条直线，使之与各训练样本数据点都比较接近，那么根据该直线，就可以计算出任意房屋面积的房屋总价了。



LinearRegression类

- Scikit-learn提供了sklearn.linear_model.LinearRegression线性回归类，可以解决大部分常见的线性回归操作

- 构造方法
- model = LinearRegression(fit_intercept = True, normalize = False, copy_X = True, n_jobs = 1)
  - ✓ fit_intercept：是否计算模型的截距，默认值是True，为False时则进行数据中心化处理
  - ✓ normalize：是否归一化，默认值是False
  - ✓ copy_X：默认True，在X.copy()上进行操作，否则会在原始数据X上进行操作，覆盖原始数据
  - ✓ n_jobs：表示使用CPU的个数，默认1，当-1时，代表使用全部的CPU

LinearRegression类

- LinearRegression类的属性和方法
  - ✓ coef_：训练后的输入端模型系数，如果label有两个，即y值有两列，是一个2D的数组
  - ✓ intercept_：截距，即公式 $f(\boldsymbol{x}) = w_1 x_1 + w_0$ 中 w_0 的值
  - ✓ fit(x, y)：拟合函数，通过训练数据x和训练数据的标签y来拟合模型
  - ✓ predict(x)：预测函数，通过拟合好的模型，对数据x预测y值
  - ✓ score：评价分数值，用于评价模型好坏

下面介绍使用 LinearRegression 类解决关于房价预测的单变量线性回归问题，先分析求解步骤， 再提供完整代码并绘图显示拟合结果

求解步骤

- 第一步：准备训练数据
  - ✓ X_train = np.array([[75], [87], [105], [110], [120]])
  - ✓ y_train = np.array([270, 280, 295, 310, 335])
- 第二步：创建模型对象
  - ✓ model = LinearRegression( )
- 第三步：执行拟合
  - ✓ model.fit(X_train, y_train)
- 第四步：准备测试数据
  - ✓ X_test = np.array([[82], [104]])
  - ✓ model.predict(X_test)

代码3.2

```
#代码3.2 基于Scikit-learn实现房价预测线性规划代码
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# 以矩阵形式表达(对于单变量，矩阵就是列向量形式
xTrain = np.array([[75], [87], [105], [110], [120]])
yTrain = np.array([ 270, 280, 295, 310, 335])
model = LinearRegression() # 创建模型对象
model.fit(xTrain, yTrain) # 根据训练数据拟合出直线(以得到假设函数)

print("截距b或w0=", model.intercept_)     # 截距
print("斜率w1=", model.coef_)          # 斜率
# 预测面积为70的房源的总价
print("预测面积为70的房源的总价：", model.predict([[70]]))
# 也可以批量预测多个房源，注意要以列向量形式表达
xTest = np.array([85, 90, 93, 109])[:, np.newaxis]
yTestPredicted = model.predict(xTest)
print("新房源数据的面积：", xTest)
print("预测新房源数据的总价：", yTestPredicted)
```

代码3.2

```
def initPlot():
    plt.figure()
    plt.title('House Price vs House Area')
    plt.xlabel('House Area')
    plt.ylabel('House Price')
    # 设置x轴和y轴的值域分别为70~130和240~350
    plt.axis([70, 130, 240, 350])
    plt.grid(True)
    return plt
plt = initPlot() yTrain
plt.plot(xTrain,, 'k.')   #格式字符串'k.'，表示绘制黑色的散点
#画出蓝色的拟合线
plt.plot([[70], [130]], model.predict([[70], [130]]), 'b-')
plt.show()
```

代码3.2

截距b或w0= 163.75113877922868

斜率w1= [1.35059217]

预测面积为70的房源的总价 [258.29259034]

新房源数据的面积： [[85]

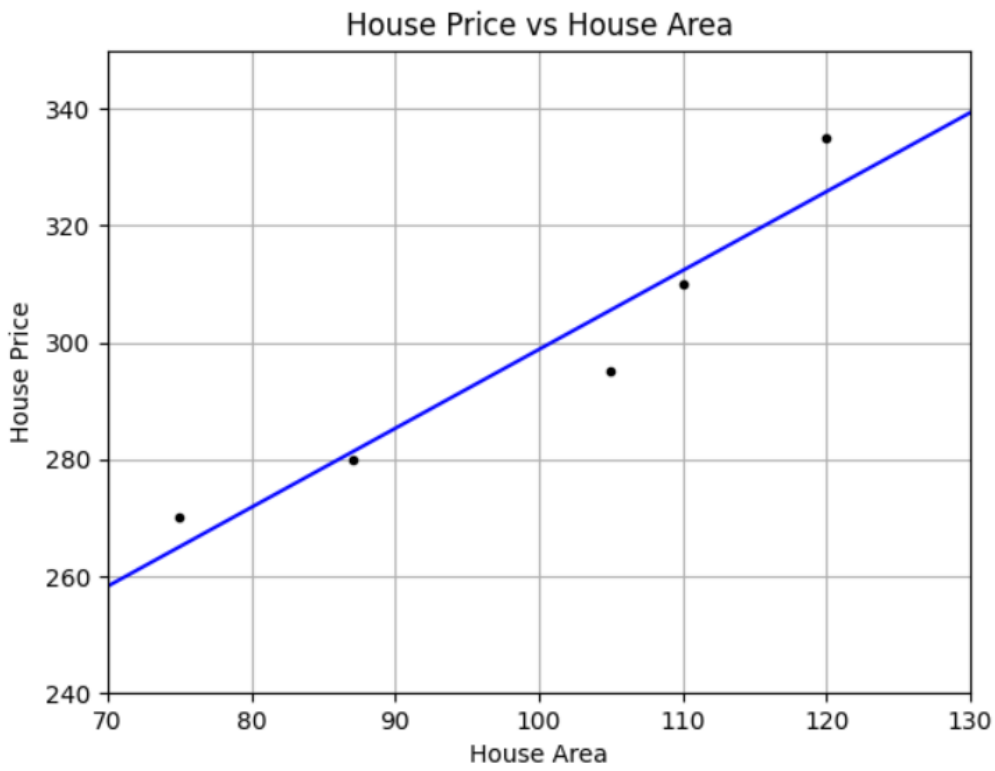
[90]

[93]

[109]]

预测新房源数据的总价：

[278.55147282 285.30443365 289.35621014 310.96568479]



思考

- 通过以下两个特征是否可以用LinearRegression实现房价预测
  - ✓ 房子长度
  - ✓ 房子宽度

南京郵電大學
Nanjing University of Posts and Telecommunications

Python编程与人工智能应用

py for AI

张伯雷
南京邮电大学，计算机学院

https://bolei-zhang.github.io/course/python-ai.html

2026/4/20

求解步骤

- 第一步：准备训练数据
  - ✓ X_train = np.array([[75], [87], [105], [110], [120]])
  - ✓ y_train = np.array([270, 280, 295, 310, 335])
- 第二步：创建模型对象
  - ✓ model = LinearRegression ( )
- 第三步：执行拟合
  - ✓ model.fit(X_train, y_train)
- 第四步：准备测试数据
  - ✓ X_test = np.array([[82], [104]])
  - ✓ model.predict (X_test)

问题2：如何优化模型?

问题1：模型结果是否准确?

模型拟合

- 使得在测试数据集中$f(X_test)$和真实的y_test尽量接近
- 悲观的是，我们并不知道y_test

- 退而求其次，希望在训练数据集中让$f(X_train)$和真实的y_train尽量接近
  - ✓ 在测试数据集中有泛化性（generalization）吗?
  - ✓ 如何定义接近?

- 对于回归问题，可以定义不同损失函数（loss function）
  - ✓ 均方误差（mean squared error）

$$mse = \sum_{i=1}^{N} (f(X_i) - y_i)^2$$

模型评价

- 如何评价该模型的好坏：
  - ✓ 用什么**数据**对拟合模型进行评价？
    - ⇢ 用训练数据计算的模型误差称之为训练误差
    - ⇢ 用测试数据计算的模型误差称之为测试误差
    - ⇢ 在训练过程中，只有训练数据是可见的
    - ⇢ 训练误差的最小化，不一定能使得测试误差最小化
  - ✓ 用什么**指标**对拟合模型进行评价？
    - ⇢ 计算线性回归误差的指标主要包括**残差平方和**与**R方**（r-squared）

残差

- 残差（Residual）
  - ✓ 观测值y与通过模型判别函数计算的y值之间的差异
  - ✓ 残差平方和是训练数据或者测试数据中所有样本残差的平方和
  - ✓ 残差值越小则对应数据的拟合度越好，当残差为0时，预测y值与观测值y完全一致

$$SS_{res} = \sum_{i=1}^{m} (f(x_i) - y_i)^2$$

R方

- R方（R-Square），又称决定系数（coefficient of determination）
  - ✓ 表达因变量与自变量之间的总体关系
  - ✓ 反映了因变量 y（标签）的波动，有多少百分比能被自变量 x（特征）的波动所描述
  - ✓ 与残差平方和在方差中所占的比率有关

$$SS_{res} = \sum_{i=1}^{m} (f(x_i) - y_i)^2 \qquad SS_{total} = \sum_{i=1}^{m} (\bar{y} - y_i)^2$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{total}}$$

- R方为负、0、1分别代表什么?

残差与R方的计算

- 手动计算：
  - ✓ 训练数据残差平方和：
    - ssResTrain=sum((yTrainPredicted - yTrain) ** 2)
  - ✓ 测试数据残差平方和：
    - ssResTest= sum((yTestPredicted - yTest) ** 2)
  - ✓ 测试数据方差：
    - ssTotalTest= sum((np.mean(yTest) - yTest) ** 2)
  - ✓ 测试数据R方：
    - rsquareTest=1 – ssResTest / ssTotalTest
- LinearRegression提供的自动计算方法：
  - ✓ 训练数据残差平方和：model._residues，通过训练数据训练模型后自动获得
  - ✓ 自动计算R方的函数：model.score(xTest, yTest)

代码3.3

```
#代码 3.3 加入评价的基于 Scikit-learn 实现房价预测线性回归代码
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

xTrain = np.array([[75], [87], [105], [110], [120]]) # 训练数据(面积)
yTrain = np.array([270,280,295,310,335]) # 训练数据(总价)
xTest = np.array([85, 90, 93, 109])[:,np.newaxis] # 测试数据(面积)
yTest = np.array([280, 282, 284, 305]) # 测试数据(总价)

model = LinearRegression()
model.fit(xTrain, yTrain)

# 针对训练数据进行预测以计算训练误差
yTrainPredicted = model.predict(xTrain)
# 针对测试数据进行预测
yTestPredicted = model.predict(xTest)
# 手动计算训练数据集残差
ssResTrain = sum((yTrainPredicted - yTrain)**2) print(ssResTrain)
# 自动计算的训练数据集残差
print(model._residues)
```

227.29

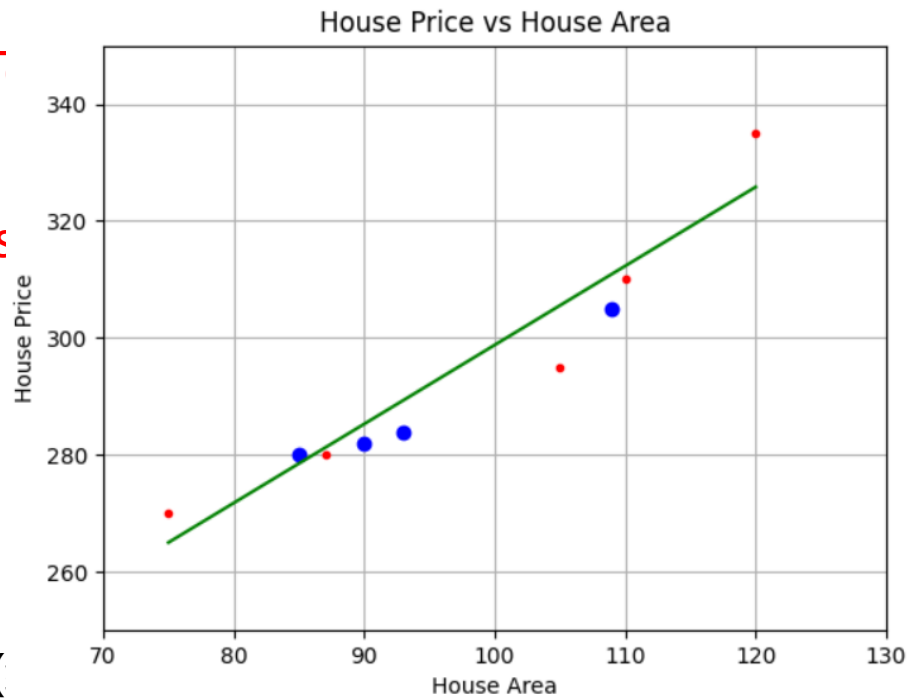
227.29

0.80

0.80

代码3.3

```
# 手动计算测试数据集残差
ssResTest = sum((yTestPredicted - yTest)**2)
# 手动计算测试数据集 y 值偏差平方和
ssTotalTest = sum((np.mean(yTest) - yT
 # 手动计算测试数据 R 方
rsquareTest = 1 - ssResTest / ssTotalTes
print(rsquareTest)
# 自动计算的测试数据集的 R 方
print(model.score(xTest, yTest))

plt.plot(xTrain, yTrain, 'r.') # 训练点数据(
plt.plot(xTest, yTest, 'bo') # 测试点数据(蓝色，大点)
plt.plot(xTrain, yTrainPredicted, 'g-') # 拟合的函数直线(绿色)
plt.show()
```



最小二乘法求解

- 根据残差的定义，单个训练数据点的残差是 $f\left(\boldsymbol{x}^{(i)}\right)-y^{(i)}$
- 训练目标是最小化训练数据残差绝对值之和

$$\sum_{i=1}^{m}\left|f\left(\boldsymbol{x}^{(i)}\right)-y^{(i)}\right|$$

- 绝对值不容易进行包括导数运算，因此采用数据的残差平方和

$$\sum_{i=1}^{m}\left(f\left(\boldsymbol{x}^{(i)}\right)-y^{(i)}\right)^{2}$$

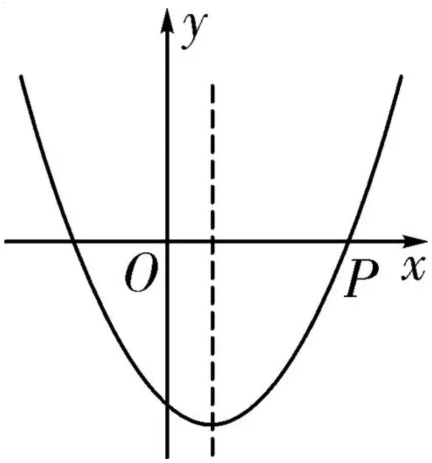
替代残差绝对值之和作为优化目标，使得残差平方和最小化，这种方法被称为**最小二乘法**

- 对最小二乘法的优化目标进行求解，有两种具体的方法，分别是**导数法**和**矩阵法**

导数法求解最小二乘法

- 训练目标：$L(w_0, w_1) = \frac{1}{m}\sum_{i=1}^{m}(f(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{m}\sum_{i=1}^{m}(w_1 x_1^{(i)} + w_0 - y^{(i)})^2$

- 函数 L 也可被称为“损失”（Loss）函数
  - ✓ 分别对 w_0 和 w_1 求一阶偏导，并使之为0



$$\begin{cases} \frac{\partial L}{\partial w_0} = 2\sum_{i=1}^{m}(w_1 x_1^{(i)} + w_0 - y^{(i)}) = 0 \\ \frac{\partial L}{\partial w_1} = 2\sum_{i=1}^{m} x_1^{(i)}(w_1 x_1^{(i)} + w_0 - y^{(i)}) = 0 \end{cases}$$

  - ✓ 求得 $w_0 = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - w_1 x_1^{(i)}) = \overline{y} - w_1\overline{x_1}$，其中 $\overline{y}$ 和 $\overline{x_1}$ 表示平均值
  - ✓ 求得

$$w_1 = \frac{\sum x_1^{(i)} y^{(i)} - m\overline{y}\,\overline{x_1}}{\sum (x_1^{(i)})^2 - m\overline{x_1}^2}$$

- 最终公式：

$$\begin{cases} w_0 = \overline{y} - w_1\overline{x_1} \\ w_1 = \frac{\mathrm{cov}(x_1, y)}{\mathrm{var}(x_1)} \end{cases}$$

代码3.4

```
#代码3.4 使用导数求解最小二乘法
import numpy as np
X_train = np.array([75, 87, 105, 110, 120])  # 训练数据(面积)
y_train = np.array([270,280,295,310,335])  # 训练数据(总价)

w1 = np.cov(X_train , y_train, ddof = 1)[1, 0] / np.var(X_train, ddof = 1)
w0 = np.mean(y_train) - w1 * np.mean(X_train)
```

w1= 1.350592165198907

w0= 163.75113877922863

使用矩阵运算求解

- 下面在使用矩阵法对最小二乘法的优化目标进行求解

$$y^{(i)} = w_0 1 + w_1 x_1^{(i)} + \cdots + w_d x_d^{(i)}$$

- 转为$\boldsymbol{y} = \mathbf{X}\mathbf{w}$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \qquad \mathbf{x}^{\mathsf{T}} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \qquad \mathbf{X}^{\mathsf{T}} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \cdots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & x_2^{(3)} & \cdots & x_2^{(m)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_d^{(1)} & x_d^{(2)} & x_d^{(3)} & \cdots & x_d^{(m)} \end{bmatrix}$$

- 求得结果$w = (\mathrm{X}^{\mathrm{T}}\mathrm{X})^{-1}\mathrm{X}^{\mathrm{T}}\mathrm{y}$ （演示推导过程）

https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

代码3.5

```
#代码3.5 求参数w的矩阵方法代码：linreg_matrix.py
import numpy as np
def linreg_matrix(x, y):
    X_X_T = np.matmul(x, x.T)
    X_X_T_1 = np.linalg.inv(X_X_T)
    X_X_T_1_X = np.matmul(X_X_T_1, x)
    X_X_T_1_X_Y_T = np.matmul(X_X_T_1_X, y.T)
return X_X_T_1_X_Y_T
```

代码3.6

```
#代码3.6 使用矩阵方法求解房价预测问题
import numpy as np
from linreg_matrix import linreg_matrix
# 训练数据(面积)，每行表示一个数据点
xTrain = np.array([[ 75 ], [ 87 ], [ 105 ], [ 110 ], [ 120 ]])
# 训练数据(总价)，每行表示一个数据点
yTrain = np.array([270, 280, 295, 310, 335])[:,np.newaxis]
def make_ext(x) : #对x进行扩展，加入一个全1的行
    ones = np.ones(1)[:, np.newaxis] #生成全1的行向量
    new_x = np.insert(x, 0, ones, axis = 0)
    return new_x
#为适应公式3.11的定义，将xTrain和yTrain
一个数据点
x = make_ext(xTrain.T)
y = yTrain.T
print("x=", x)
print("y=", y)
w = linreg_matrix(x, y)
print("w=", w)
```

```
x= [[  1   1   1   1   1]
 [75  87 105 110 120]]
y= [[270 280 295 310 335]]
w= [[163.75113878]
 [1.35059217]]
```

思考

- 当样本数量远大于特征数量时，例如100万个样本，1000维特征，用最小二乘法求解是否合适?

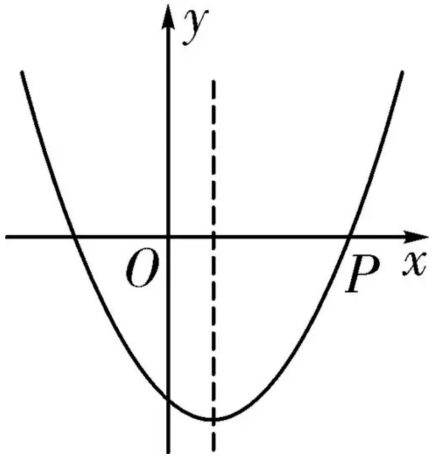
- 当特征数量远大于样本数量时，例如1000个样本，100万维特征，是否可以用最小二乘法?

模型优化

- 在线性回归问题中，当优化目标是均方误差时，可以有**闭式解**

$$\mathbf{w}^* = \left(\mathrm{X}^{\mathrm{T}}\mathrm{X}\right)^{-1}\mathrm{X}^{\mathrm{T}}\mathrm{y}$$

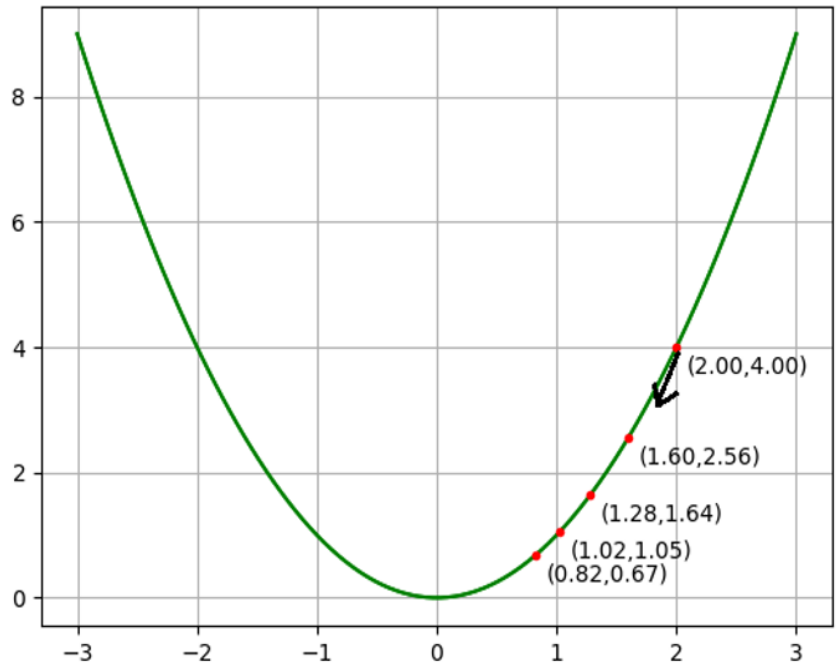
- 但是在实际情况中
  - ✓ 闭式解的计算开销往往很高
    - ⇢ 假设数据包含m=1000个样本
    - ⇢ 每个样本有d=100万个特征
    - ⇢ 矩阵相乘：d*m*d
    - ⇢ 矩阵求逆：d*d*d
  - ✓ $X^T X$不是满秩矩阵，需要有正则项
  - ✓ sklearn.linear_model.SGDRegressor



- **梯度下降法**：在一个随机的位置沿着梯度的负方向，也就是当前最陡峭的位置向下走一步，然后继续求解当前位置梯度
- 牛顿法：在极小点附近通过二阶泰勒展开，找到极小点的下一个估计值

二次函数梯度下降法求解

- 梯度下降法是一种求函数极值的数值方法，其基本思想是对目标函数选取一个初始点，从该点出发，根据梯度（导数）的方向步步运算，能够到达一个极值，该极值可能是局部最优解



- 可能输出不精确的极值；
- 如果有多个极值，则有可能找到的是局部最优点，而非全局最优点

二次函数梯度下降法求解

- 梯度下降法是一种**求函数极值的数值方法**，其基本思想是对目标函数选取一个初始点，从该点出发，根据梯度（导数）的方向步步运算，能够到达一个极值，该极值**可能是局部最优解**
- 对于目标损失函数$f(w) = w^2$的梯度下降法求极值
  - ✓ 第一步：选定一个初始值，比如选定 $w_{(0)} = 2$，$f(w_{(0)}) = 4$
  - ✓ 第二步：计算该点的导数 $f'(w_{(0)}) = 2w_{(0)}$
  - ✓ 第三步：按公式计算 w 的新值 $w_{(1)}$

$$w_{(1)} = w_{(0)} - l_r f'(w_{(0)})$$

  - ⇢ $l_r = 0.1$ 是学习速度
  - ⇢ $f'(w)$ 用于控制每步前进的方向，朝着导数相反方向前进
- 第四步：计算$f(w_{(1)})$，根据规则判断是否收敛，比如两者差值的绝对值小于某个临界值（如0.00001）
- 第五步：判断是否达到指定的循环次数（如1000次），如果达到则结束，如果未达到则设 $w_{(0)} = w_{(1)}$，跳转到第二步继续执行

为什么选择梯度的方向？

算法：通用下降方法

1. 输入：一个初始点$x^{(0)} \in \mathrm{dom} f$
2. 重复：
3. 　　找到一个下降的方向$\Delta x^{(k)}$
4. 　　选择一个步长 $\alpha^{(k)}$
5. 　　更新：$x^{(k+1)} = x^{(k)} + \alpha^{(k)} \Delta x^{(k)}$
6. Until：满足停止条件

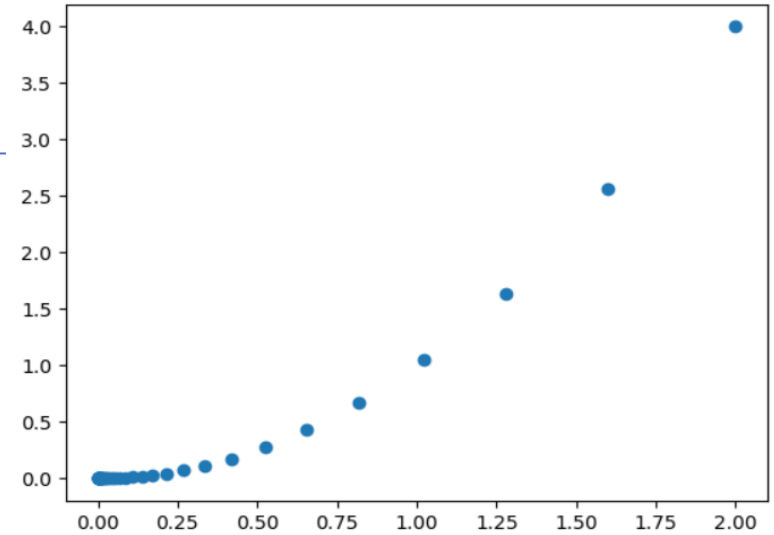
代码3.7

```
import numpy as np
import matplotlib.pyplot as plt
def obj_fun(x):    return x * x  #需要求极值的目标函数
def dir_fun(x):    return 2 * x  #目标函数的导数
x_list = [ ]
y_list = [ ] # 用于保存经过的点
def minimize(init_x, lr = 0.1, diff = 1e-9, max_iter = 1000):
    # init_x: 初始点
    # lr: 学习速率，dif: 相邻两步差异临界值，max_iter: 最大迭代次数
    x0 = init_x
    y0 = obj_fun(x0)
    x_list.append(x0)
    y_list.append(y0)
    for i in range(max_iter):
        x1 = x0 - lr * dir_fun(x0)
        y1 = obj_fun(x1)
        x_list.append(x1)
        y_list.append(y1)
```

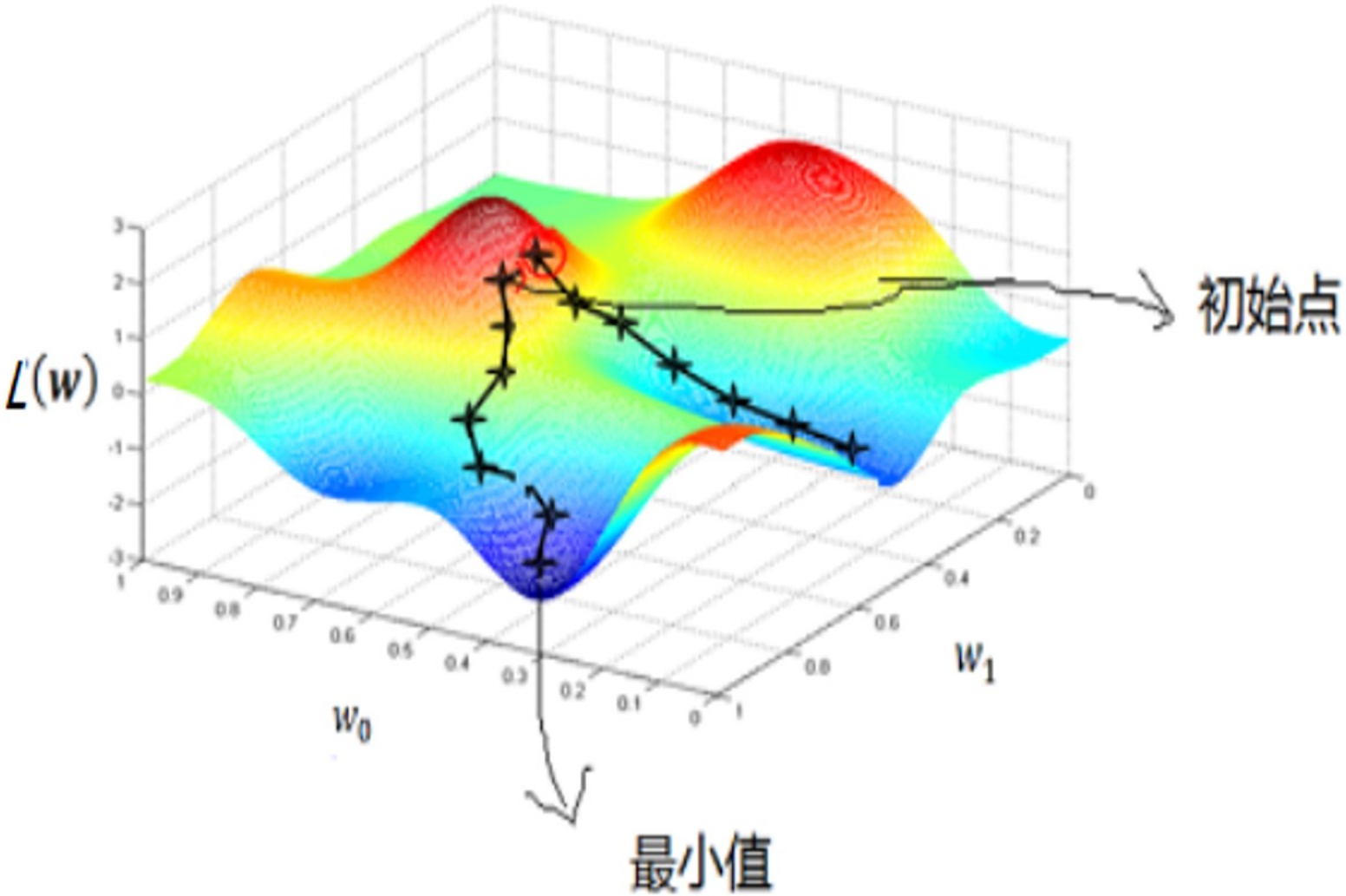
代码3.7

```
    if abs(y1 - y0) <= diff: #达到收敛条件
        print("是否收敛：True", "极小点：(%e, %e)"%(x1, y1), "循环次数：%d"%i)
        return
    x0 = x1
    y0 = y1
  print("是否收敛：False", "极小点：NaN", "循环次数：%d" % max_iter)
minimize(2.0)
plt.plot(x_list, y_list, "o") #绘制过程点
plt.show()
```

是否收敛： True 极小点：(3.794275e-09, 1.439652e-17) 循环次数：89



多参数随机梯度下降



批量梯度下降法

- 对于最小二乘法，其目标函数是：

$$L(w_0, w_1) = \frac{1}{2m}\sum_{i=1}^{m}(f(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{2m}\sum_{i=1}^{m}(w_1 x_1^{(i)} + w_0 - y^{(i)})^2$$

- ✓ 优化的参数有两个，分别是 w_0 和 w_1
- ✓ 需要一次性批量地使用全部训练数据，被称为批量梯度下降法（Batch Gradient Decent）

算法步骤

- 算法步骤：
  - ✓ 第一步：随机选定 w_0 和 w_1 的初始值，并计算 $L(w_0, w_1)$
  - ✓ 第二步： w_0 和 w_1 的偏导函数分别为$\partial L/(\partial w_0)$和$\partial L/(\partial w_1)$，偏导方向 w 变化最快

$$w_0 = w_0 - lr * \frac{\partial L}{\partial w_0} \qquad w_1 = w_1 - lr * \frac{\partial L}{\partial w_1}$$

⇢ 利用更新后的 w_0 和 w_1 值再次计算 $L(w_0, w_1)$ 和其导数

  - ✓ 第三步：循环执行第二步，**直到参数变化小于某个临界值或循环迭代次数达到设定的最大次数**
- 演示推导过程

$$\frac{\partial L(w_0, w_1)}{\partial w_0} = \frac{1}{m}\sum_{i=1}^{m}(w_1 x_1^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial L(w_0, w_1)}{\partial w_1} = \frac{1}{m}\sum_{i=1}^{m}[(w_1 x_1^{(i)} + w_0 - y^{(i)}) \bullet x_1^{(i)}]$$

代码3.8 批量梯度下降实现

```
#代码3.8 批量梯度下降法：bgd_optimizer.py
def bgd_optimizer(target_fn, grad_fn, init_w, X, Y, lr=0.0001,tolerance=1e-12, max_iter=100000000):
    w = init_w
    target_value = target_fn(w, X, Y)          # 计算当前w值下的L(w)值

    for i in range(max_iter):
        grad = grad_fn(w, X, Y)                # 计算梯度
        next_w = w - grad * lr                 # 向量计算，调整了w
        next_target_value = target_fn(next_w, X, Y)    # 计算新值

        # 如果两次计算之间的误差小于tolerance，则表明已经收敛
        if abs(next_target_value - target_value) < tolerance:
            return i, next_w #返回迭代次数和参数w的值
        else: w, target_value = next_w, next_target_value    # 继续进行下一轮计算

    return i, None #返回迭代次数，由于未收敛，w没有优化的值
```

代码3.9

```
#代码3.9 基于梯度下降法求解房价预测问题
import numpy as np
from bgd_optimizer import bgd_optimizer
import matplotlib.pyplot as plt
def target_function(w, X, Y):  # 定义目标函数
    w0, w1 = w   # w:[w0, w1]
    # 应使用np.sum，而不要使用sum。
    #np.sum支持向量/矩阵运算
    return np.sum((w0 + X * w1 – Y) ** 2) / (2 * len(X))

# 根据目标函数定义梯度，对x0和x1求导数，
# 累计各点导数平均值
def grad_function ( w, X, Y ):
    w0, w1 = w
    # 对应w0的导数
    w0_grad = np.sum(w0 + X * w1 – Y) / len(X)
     # 对应w1的导数。注意采用向量运算
    w1_grad = X.dot(w0 + X * w1 – Y) / len(X)
    return np.array([w0_grad, w1_grad])
```

代码3.9

```
# 训练数据(面积)
x = np.array([75, 87, 105, 110, 120], dtype = np.float64 )
# 训练数据(总价)
y = np.array([270, 280, 295, 310, 335], dtype = np.float64)
np.random.seed(0)
init_W = np.array([np.random.random(), np.random.random()]) # 随机初始化W值
i, W= bgd_optimizer(target_function, grad_function, init_W, x, y)
if W is not None :
    w0, w1 = w
    print("迭代次数：%d，最优的w0和w1:(%f, %f)" % (i, w0, w1))
else: print ( "达到最大迭代次数，未收敛" )
```

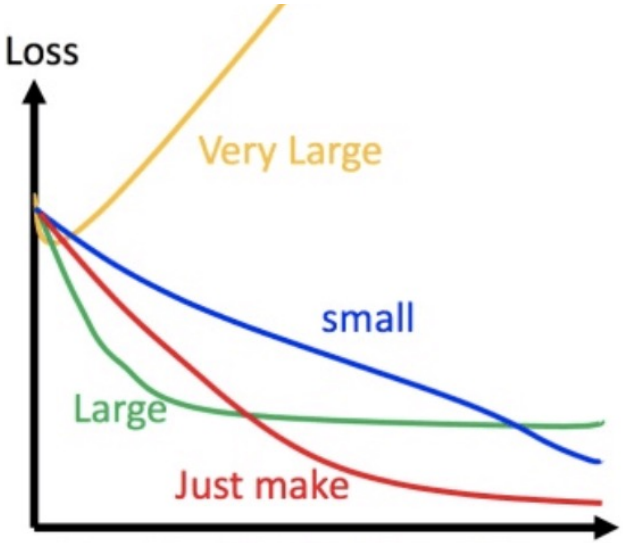
迭代次数：4051854，

最优的w0和w1:(163.746743, 1.350635)

思考：如果将lr设为较大值（如设置为0.01或0.1）会出现什么状况?
如果将lr设为较小值（如设置为1e-6）呢?

参数的选择

- 学习率lr的选择



- w初始值的选择
  - ✓ 不要距离目标值太远

南京郵電大學
Nanjing University of Posts and Telecommunications

Python编程与人工智能应用

py for AI

张伯雷
南京邮电大学，计算机学院

https://bolei-zhang.github.io/course/python-ai.html

2026/4/20

随机梯度下降法

- 批量梯度下降法的缺陷：当样本数量非常庞大时，计算复杂、费时且极有可能超出硬件能力（内存容量、CPU计算能力等）限制。

- 随机梯度下降法（Stochastic Gradient Decent，简称SGD），**其原理是每次随机选取一部分样本对目标函数进行优化。**

- 随机梯度下降法能够达到不弱于批量梯度下降法的优化效果，在实际场景中应用更加广泛。

代码3.10

```
#代码3.10 随机梯度下降法：sgd_optimizer.py
import numpy as np
def sgd_optimizer(target_fn,grad_fn, init_w, X, Y, lr=0.0001, tolerance=1e-12, max_iter=1000000000):
    w, rate = init_w, lr
    min_w, min_target_value = None, float("inf")
    no_improvement = 0
    target_value = target_fn(W, X, Y)
    for i in range(max_iter):
        index = np.random.randint(0, len(X))          # 获得一组随机数据的索引值
        gradient = grad_fn(w, X[index], Y[index])     # 计算该数据点处的导数
        w = w - lr * gradient
        new_target_value = target_fn(w, X, Y)
        if abs(new_target_value - target_value) < tolerance :
            return i, w
    return i, None
```

批量梯度下降vs随机梯度下降

- 运算空间
  - ✓ 批量：需要所有样本，在样本很大时，运算空间要求很高
  - ✓ 随机：每一轮只需要一个样本，单步更新快，需要的空间少

- 算法稳定性
  - ✓ 批量：通过所有样本计算梯度，更稳定
  - ✓ 随机：需要通过每个样本计算梯度，不稳定

多变量线性回归问题

- 现实生活中，房价不仅与面积有关，也与户型、楼层等相关。增加了房屋户型作为因变量，变化后的数据如表所示
  - ✓ 户型类型：1（一居室）、2（二居室）、3（三居室）、4（四居室）

训练样本	房屋面积（平方米）	房屋户型	房屋总价（万元）
1	75	1	270
2	87	3	280
3	105	3	295
4	110	3	310
5	120	4	335

验证样本	房屋面积（平方米）	房屋户型	房屋总价（万元）
1	85	2	280
2	90	3	282
3	93	3	284
4	109	4	305

基于Scikit-learn库求解

```
#代码3.12 使用Scikit-learn库实现多变量房价预测问题求解
import numpy as np
from sklearn.linear_model import LinearRegression
xTrain = np.array([[75, 1], [87, 3], [105, 3], [110, 3], [120, 4]])  # 训练数据(面积，户型)
yTrain = np.array([270,280,295,310,335])            # 训练数据(总价)
xVal = np.array([[ 85, 2], [90, 3], [93, 3], [109, 4]]) # 验证数据(面积，户型)
yVal = np.array([280, 282, 284, 305])               #验证数据(总价)
model = LinearRegression()
model.fit(xTrain, yTrain)
print(model._residues)                              #训练数据
print(model.score(xVal, yVal))                      #验证数
```

残差：226.82910636037715
R方：0.8373813078020356

基于最小二乘法求解

```
#代码3.13 使用矩阵方法求解多变量房价预测问题
import numpy as np
from linreg_matrix import linreg_matrix
xTrain = np.array([[ 75, 1], [87, 3], [105, 3], [110, 3], [120, 4]])  # 训练数据(面积，户型)
yTrain = np.array([270, 280, 295, 310, 335]) # 训练数据(总价)
xTest = np.array([[ 85, 2], [90, 3], [93, 3], [109, 4]]) # 验证数据(面积，户型)
yTest = np.array([280, 282, 284, 305])   # 验证数据(总价)
def make_ext(x): #对x进行扩展，加入一个全1的行
    ones = np.ones(1)[:, np.newaxis] #生成全1的行向量
    new_x = np.insert(x, 0, ones, axis = 0)
    return new_x
x = make_ext(xTrain.T)
y = yTrain.T
```

基于最小二乘法求解

```
w = linreg_matrix(x, y)
print("w=", w)
yTrainPredicted = w.dot(x)
yTestPredicted = w.dot(make_ext(xTest.T))
ssResTrain = sum((yTrain – yTrainPredicted )**2)
print("训练数据残差平方和：", ssResTrain))
ssResTest = sum((yTest – yTestPredicted)**2
ssTotalTest = sum((yTest - np.mean(
rsquareTest = 1 - ssResTest / ssTotal
print("验证数据R方：", rsquareTest)
```

```
w=  [162.19293587        1.38427832        -0.63935732]
训练数据残差平方和：  226.8291063603765
验证数据R方：  0.837381307801918
```

基于梯度下降法求解

- 梯度下降方法同样可以用于多变量房价预测求解，需要对目标函数和梯度函数做一些扩展。

$$L(\mathbf{w})=\frac{1}{2m}\sum_{i=1}^{m}(\sum_{k=1}^{d}w_k x_k^{(i)}+w_0-y^{(i)})^2$$

$$\frac{\partial L(\mathbf{w})}{\partial w_0}=\frac{1}{m}\sum_{i=1}^{m}(\sum_{k=1}^{d}w_k x_k^{(i)}+w_0-y^{(i)})$$

- 对于 $j>0$，有

$$\frac{\partial L}{\partial w_j}=\frac{1}{m}\sum_{i=1}^{m}\left[(\sum_{k=1}^{d}w_k x_k^{(i)}+w_0-y^{(i)})x_j^{(i)}\right]$$

代码3.14

```
#代码3.14 基于批量梯度下降法求解多变量房价预测问题
def target_function ( w, X, Y ) :  # 定义目标函数
    return np.sum ( ( w [ 0 ] + X.dot ( w [ 1: ]) – Y ) ** 2 ) / ( 2 * len ( X ) )

def grad_function(w, X, Y): # 根据目标函数定义梯度，对w0, w1, w2求导数平均值
    w0_grad = np.sum(w 0] + X.dot(w [1: ]) – Y) / len(X) #对应w0的导数
    #对应w1的导数
    w1_grad = X[:, 0].dot(np.array(w[0]) + X.dot( w [1: ] ) – Y ) / len ( X )
    #对应w2的导数
    w2_grad = X[:, 1].dot(np.array(w[0]) + X.dot( w [1: ]) – Y ) / len (X)
    return np.array ( [ w0_grad,w1_grad,w2_grad ] )
```

代码3.14

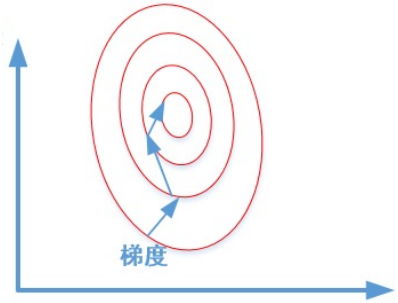
```
x = np.array ( [ [ 75, 1 ], [ 87, 3 ], [ 105, 3 ], [ 110, 3 ], [ 120, 4 ] ], dtype = np.float64 )
y = np.array ( [ 270, 280, 295, 310, 335 ], dtype = np.float64 )

np.random.seed ( 0 )
init_W = np.random.random(3)   #随机初始化W
i, w = bgd_optimizer(target_function, grad_function, i
if W is not None :
    w0, w1, w2 = w
    print ( "迭代次数：%d，最优的w0, w1, w2:(%f, %f, %f)" % ( i, w0, w1, w2 ) )
else: print ( "达到最大迭代次数，未收敛" )
```

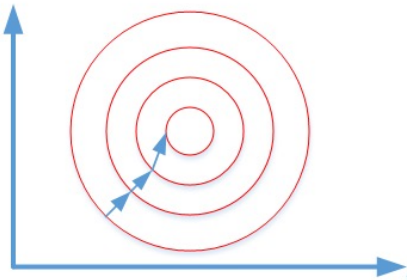
迭代次数：6598762，最优的w0, w1, w2:(162.185449, 1.384389, -0.640646)

数据归一化问题

- 在多变量情况下，各变量的值域有很大差别，比如房屋面积的范围是几十到几百的浮点数，房屋户型的值域是1到5之间的整数
- **值域差异过大，容易造成计算过程中出现溢出或无法收敛，导致各个变量作用权重受到影响**
- 可以对数据进行**归一化（Normalization）**处理来解决这一问题
- 归一化方法：

$$x_norm_i = \frac{x_i - \overline{x_i}}{std(x_i)}$$





代码3.15

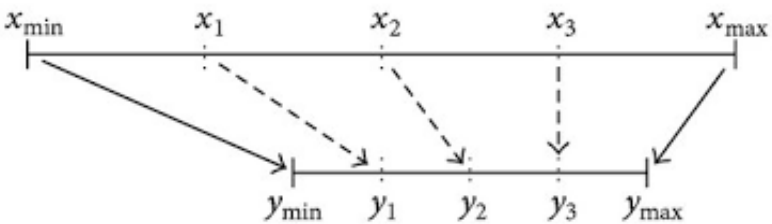
```
#代码3.15 先进行归一化处理，再使用Scikit-learn库求解
import numpy as np
from sklearn.linear_model import LinearRegression
def normalize ( X ) :
    X_mean = np.mean ( X, 0 ) #计算均值
    X_std = np.var ( X, 0 ) #计算标准差
    return ( X - X_mean ) / X_std
xTrain = np.array([[75, 1], [87, 3], [105, 3], [110, 3], [120, 4]])  # 训练数据(面积，户型)
yTrain = np.array([270, 280, 295, 310, 335])          # 训练数据(总价)
xTrain = (normalize(xTrain))
model = LinearRegression()
model.fit(xTrain, yTrain)
print(model._residues)                                # 训练数据集残差
```

输出结果为：226.82910636037712

数据归一化

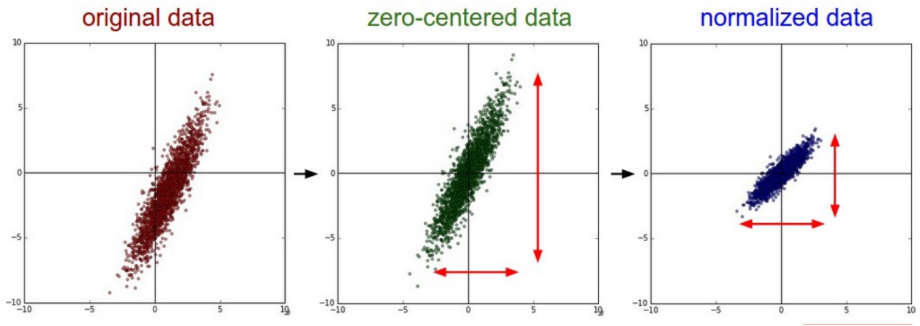
- 两种常见的归一化方法
  - ✓ Min-max

$$\frac{x - x_{min}}{x_{max} - x_{min}}$$



  - ✓ Z-score

$$\frac{x - x_{mean}}{x_{std}}$$



- 思考：在进行预测时，是否仍然需要归一化

高阶拟合问题

- 单变量房价预测问题中只有一个自变量“房屋面积”，因此拟合出来的是一条直线；多变量房价预测问题中有两个自变量“房屋面积”和“房屋户型”，因此拟合出来的是一个直平面

- 采用“曲线”或“曲面”模型来拟合能够对训练数据产生更逼近真实值的效果，这就是高阶拟合，有可能是非线性的。

高阶拟合问题

- 对于一个自变量的场景，可以采用多阶函数：

$$f_{\mathrm{n}}(x)=\sum_{k=0}^{n} w_k x^k$$

- 二阶单变量函数为： $f_2(x)=w_0+w_1x+w_2x^2$
- 三阶单变量函数为： $f_3(x)=w_0+w_1x+w_2x^2+w_3x^3$
- 四阶单变量函数为： $f_4(x)=w_0+w_1x+w_2x^2+w_3x^3+w_4x^4$
- 可以通过如下方式进行变换：
  - ✓ 加入新变量 x_1，设 $x_1=x$；
  - ✓ 加入新变量 x_2，设 $x_2=x^2$；
  - ✓ 加入新变量 x_3，设 $x_3=x^3$；
  - ✓ 加入新变量 x_4，设 $x_4=x^4$；

代码3.16

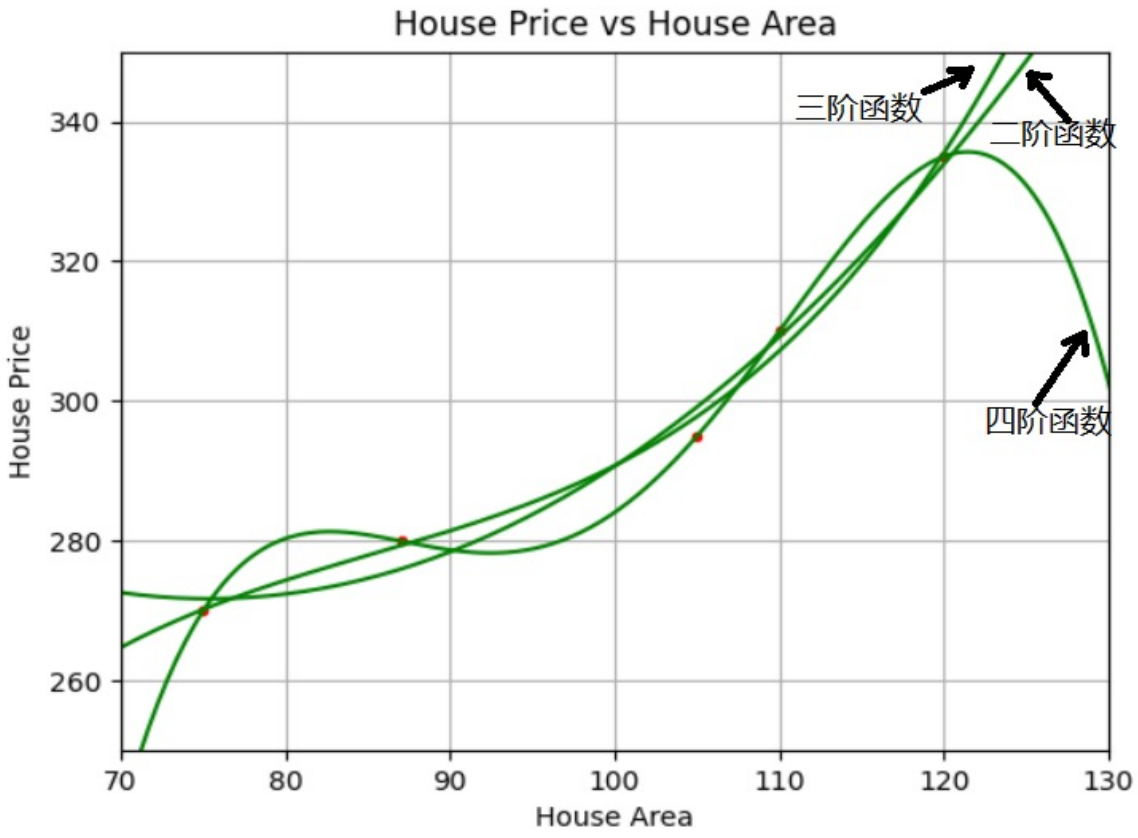
```
#代码3.16 使用Scikit-learn库实现高阶拟合单变量房价预测
xTrain = np.array([[75], [87], [105], [110], [120]])
x1 = xTrain
x2 = xTrain ** 2
x3 = xTrain ** 3
x4 = xTrain ** 4
new_xTrain2 = np.concatenate([x1, x2], axis = 1)
new_xTrain3 = np.concatenate([x1, x2, x3], axis = 1)
new_xTrain4 = np.concatenate([x1, x2, x3, x4], axis = 1)

yTrain = np.array([270, 280, 295, 310, 335])
xTest = np.array([[85 ], [90], [93], [109]])
x1 = xTest
x2 = xTest ** 2
x3 = xTest ** 3
x4 = xTest ** 4
new_xTest2 = np.concatenate([x1, x2], axis = 1)
new_xTest3 = np.concatenate([x1, x2, x3], axis = 1)
new_xTest4 = np.concatenate([x1, x2, x3, x4], axis = 1)
```

代码3.16

```
new_xTest4 = np.concatenate([ x1, x2, x3, x4 ], axis = 1)
yTest = np.array([ 280, 282, 284, 305])        # 测试数据(总价)
model1 = LinearRegression ( )
model1.fit(xTrain, yTrain)
print("一阶训练数
print("一阶测试数
model2 = Linear
model2.fit(new_x
print("二阶训练数
print("二阶测试数                                                    阶测试数
据集的R方
model3 = Linear
model3.fit(new_x
print("三阶训练数
print("三阶测试数
model4 = LinearRegression()
model4.fit(new_xTrain4, yTrain)
print("四阶训练数据集残差：", model4._residues)
print("四阶测试数据集R方：", model4.score(new_xTest4, yTest))
```

```
一阶训练数据集残差：  227.2965381111449
一阶测试数据集R方：  0.8090280549127149
二阶训练数据集残差：  38.38456969539346
二阶测a试数据集R方：  0.8714891148616254
三阶训练数据集残差：  16.488230700095926
三阶测试数据集R方：  0.9885676670452613
四阶训练数据集残差：  5.498381308907731e-18
四阶测试数据集R方：  0.8830805089175938
```

绘制图形


House Price vs House Area
三阶函数
二阶函数
四阶函数
House Price
House Area
340
320
300
280
260
70
80
90
100
110
120
130

回顾整个过程

- 问题建模
  - ✓ 回归 ...
- 收集数据
  - ✓ 回归：特征数据X，连续数据y
- 特征预处理
  - ✓ 归一化
  - ✓ 类别特征
  - ✓ 时间特征
  - ✓ 图像数据、序列数据、图结构数据
- 构建模型
  - ✓ 模型选择：线性回归
  - ✓ 损失函数：均方误差
- 模型验证&参数调优
  - ✓ 使用训练数据训练模型，使用验证数据进行参数调优
  - ✓ 验证指标：回归（wmape、R2、均方误差）
- 模型上线/AB测试
  - ✓ 在测试数据上进行模型测试（测试数据和训练数据来自于同一分布）

总结

- 线性回归定义
- 求解线性回归的多种方法
  - ✓ Scikit-Learn库函数求解法
  - ✓ 最小二乘法
  - ✓ 梯度下降法
- 多变量线性回归问题
  - ✓ 数据归一化
  - ✓ 高阶拟合

- 作业：Spoc第三章

独立同分布

- **独立同分布**”，即independent and identically distributed, 简称为 i.i.d.
- **独立性（Independent）**：数据集中的每个样本与其他样本无关，不存在隐藏的依赖关系。例如，同一个患者在不同时间的医疗记录可能相关，违背独立性。
- **同分布（Identically Distributed）**：所有样本来自同一概率分布。例如，训练集中的图片和测试集图片应具有相似的光照、角度等特征分布。

为什么需要独立同分布

- 为什么需要i.i.d.?
  - 统计学习理论（如**大数定律**和**中心极限定理**）成立的前提。
  - 模型评估的可靠性：只有当训练和测试环境一致时，测试结果才能反映真实性能。
- 如果违背独立同分布
  - 泛化能力评估失效：模型在训练集上达到95%准确率，测试集却只有50%。
  - 数据泄漏（Data Leakage）：测试集信息意外进入训练过程。
    分布偏移：
    - 协变量偏移：输入特征分布变化。*示例*：训练数据是夏季商品销售记录，测试数据包含圣诞节促销，模型无法预测节日爆款。
    - 标签偏移：输出分布变化。*示例*：疫情前训练的疾病诊断模型，在疫情爆发时因阳性样本比例剧增而失效。
  - 理论保证失效
    - 经验风险最小化（ERM）的泛化误差上界依赖于i.i.d.假设。若数据不独立或非同分布，VC维等理论工具可能无法应用。

如何保证独立同分布

数据划分

场景	推荐方法	工具示例
普通分类/回归任务	随机分层抽样（Stratified Split）	`sklearn.model_selection.train_test_split`
时间序列	按时间顺序划分（前80%训练，后20%测试）	`sklearn.model_selection.TimeSeriesSplit`
空间数据	地理区块划分（避免邻近点泄漏）	自定义区域掩码
多模态数据	模态对齐划分（确保各模态数据来源一致）	元数据匹配

一致性检测

- **统计检验**：
  - Kolmogorov-Smirnov检验（连续特征）
  - 卡方检验（离散特征）
  - 对抗验证（Adversarial Validation）：训练分类器区分训练集和测试集，若AUC接近0.5则分布相似。
- **可视化工具**